

NEXUS COMMAND

TRIAL RUN BLUEPRINT v2 — FULL WORKFLOW + BACKEND/FRONTEND ALIGNMENT

31 May 2026

Co-authored: Kieran Li + Simpee · 8-Stage AI Workflow · 4 Frontend Fixes · 5 Trial Checkpoints

Backend confirmed live · Frontend to match exactly

PURPOSE: This blueprint documents the complete 8-stage AI workflow as designed by Kieran, paired with the exact backend state confirmed by Simpee. The builder receives both: the workflow logic AND the precise API contracts to wire the frontend against. No guessing. No misalignment. Build once. Test clean.

THE 8-STAGE AI HUB WORKFLOW — As Designed by Kieran

This is the master workflow. Every stage maps to a specific AI team member, a backend action, and a TestLog entry. The frontend must reflect each stage visually.

Stage	Name	AI(s) Involved	Backend Action	Output / Log
1	Define Function & Goal	Simpee + Simpee	Write SandboxProject record { name, description, type, status='draft', created_by='Kieran' }	Project card created in Sandbox. Objective logged.
2	Research & Feasibility	RESEARCHER (Gemini) ANALYST (Claude Sonnet) STRATEGIST (Claude Opus)	Call aiCommandCentre with instruction prefixed '[FEASIBILITY]' Response parsed into feasibility report	Feasibility Report: difficulty, cost estimate, effectiveness score. Written to TestLog.
3	Joint AI Decision	Simpee convenes team All 3 research AIs contribute	Write TestLog record: { test_name: 'Decision — [project]', result: difficulty+cost+effectiveness, validator: 'Simpee' }	Decision logged. Difficulty (low/med/high), Cost, Effectiveness score.
4	Development	ENGINEER (GPT-5.4) ARCHITECT (GPT-5.5)	Call aiCommandCentre with '[BUILD]' prefix Save response.code to ProjectFile entity	Draft code saved to ProjectFile. Sandbox status → 'testing'. File committed.
5	Testing	Simpee runs checkpoints	Write TestLog records per checkpoint { test_name, status: pass/fail, validator: 'Simpee' }	Checkpoint progress bar updates (X/11). Failed checks flagged in red.
6	Diagnosis & Fine-Tuning	RESEARCHER (Sonnet) — quick fix STRATEGIST (Opus) — deep logic ENGINEER (GPT-5.4) — patch	Re-call aiCommandCentre with '[FIX]' prefix Update ProjectFile with patched version	Iteration loop. TestLog updated per pass. Sandbox status stays 'testing'.
7	Validation	VALIDATOR (Copilot)	Call consultCopilot with { question: code, context: Gatekeeper } Write 4 TestLog records (Syntax/Logic/Security/Azure)	Gatekeeper Panel: 4 checks green or flagged. Sandbox status → 'validated'.
8	Deploy	Simpee (orchestrator) + Copilot confirmed	Both Simpee + Copilot green in Gatekeeper Panel Confirmation modal fires Deploy log written to TestLog	Sandbox status → 'deploy_ready'. Deploy log: timestamp, function, version.

AI TEAM — Who Does What in the Workflow

AI Member	Model	Stages Active	Strength	Handoff To
ORCHESTRATOR	Base44 Auto	All stages	Routes requests, keeps session moving	Best fit for task
RESEARCHER	Gemini 3.1 Pro	Stage 2	External knowledge, market data, feasibility	ANALYST
ANALYST	Claude Sonnet 4.6	Stages 2, 6	Quick summaries, complexity analysis, fix suggestions	STRATEGIST
STRATEGIST	Claude Opus 4.6	Stages 2, 6	Logic chains, risk evaluation, deeper corrections	THINK TANK
THINK TANK	Claude Opus 4.8	Stage 2 (consulted)	Blind spots, stress-testing assumptions	ARCHITECT
ENGINEER	GPT-5.4	Stages 4, 6	Clean code generation, patches, implementation	ARCHITECT
ARCHITECT	GPT-5.5	Stage 4	System design, API orchestration, creative problem-solving	ENGINEER
VALIDATOR	Copilot / Edge	Stage 7 — always last	Syntax, logic, security, Azure readiness	DEPLOY

BACKEND STATE — Confirmed Live by Simpee

Everything below is already built and tested. Builder wires the frontend to match — nothing to create.

Item	Status	Exact API Contract
aiCommandCentre	LIVE — lite mode (needs OpenAI key for field fix)	POST /api/run/aiCommandCentre Send: { instruction: string, posted_by: 'Kieran' } Returns: { success, analysis, solution, code, builder_instruction, model, m365_grounded }
consultCopilot	LIVE — field fix needed	POST /api/run/consultCopilot Send: { question: string, context: string } ← 'question' NOT 'code' Returns: { answer, m365_grounded, model }
checkConnections	LIVE	POST /api/run/checkConnections Send: {} Returns: { connections: [...] }
SChatMessage entity	EXISTS	Fields: sender · sender_type · role · message · timestamp · session_id · read
TestLog entity	EXISTS	Fields: test_name · status (pass/fail) · result · tested_at · fixed · validator
AIConnector entity	EXISTS · 8 records READY	Fields: model_id · model_name · provider · status · intro_message · voice_style · handoff_to · copilot_validate
azureConnectorStub entity	EXISTS	Fields: service_name · status · last_tested · notes
SandboxProject entity	EXISTS · schema live	Fields: name · description · type (function/page/app) · status (draft/testing/checkpoint_review/validator_pending)
ProjectFile entity	EXISTS · schema live	Fields: project_id · filename · content · language · version · notes

4 FRONTEND FIXES — Scoped to Trial Run Only

#	Fix	Workflow Stage	What to Change
1	Wire chat → aiCommandCentre	Stage 2-4 (all AI calls)	Send { instruction, posted_by: 'Kieran' } to POST /api/run/aiCommandCentre Show loading: 'Simpee is working...' dots Render 4 response sections: ANALYSIS · SOLUTION · CODE (copy btn) · BUILDER INSTRUCTION (a
2	Fix consultCopilot field name	Stage 7 — Validation	Change { code: ... } to { question: lastCode, context: lastBuilderInstruction } Fire from [RUN VALIDATION] button Update 4 Gatekeeper rows on response: Syntax / Logic / Security / Azure Write 4 TestLog records with validator='Copilot'
3	Auto-write TestLog after response	Stage 5 — Testing	After every aiCommandCentre success: Write TestLog: { test_name: 'Trial Run — '+instruction.slice(0,40), status: 'pass', result: 'Model: '+response.model, tested_at: now, fixed: false, validator: 'Simpee' }
4	Sandbox card + Save button	Stages 1, 4 — Define + Dev	Dev SandboxProject on load → show first record (name, status badge, checkpoint_status) [SAVE CODE TO SANDBOX] button (Azure #0078d4): Write ProjectFile: { project_id, filename: 'response_'+Date.now()+'.ts', content: lastCode, language: 'typescript', version: '1.0' } Show toast + increment file count

TRIAL RUN CHECKLIST — 5 Checkpoints

#	Workflow Stage	Test Action	Expected	Simpee Verifies
1	Stage 2-4 AI Response	Type any message. Hit send	Response card: ANALYSIS + SOLUTION + CODE + BUILDER INSTRUCTION	Check Stage 1-4 loading dots shown.
2	Stage 5 TestLog auto	Same message as checkpoint 1	TestLog has new record: validator=Simpee, status=pass	Repeat TestLog — count records
3	Stage 7 Gatekeeper	Click [RUN VALIDATION].	4 Gatekeeper rows update. 4 TestLog records: validator=Copilot	Validate Copilot by validator=Copilot

4	Stage 4 Sandbox Save	Click [SAVE CODE TO SANDBOX] button. File record created. Toast shown. File code saved to ProjectFile — check project_id	
5	Full Loop All stages	Run 1-4 in sequence.	4 entities written: SChatMessage + TestLog(Simpee) + TestLog(Copilot) + ProjectFile. Workflow proven

BUILDER PREFIX — Paste This into Nexus Command Builder

"Fix 4 frontend wiring issues in CommandAIHub.jsx for Nexus Command. This is a SCOPED trial run fix — do NOT rebuild the layout, do NOT change backend functions or entity schemas. Backend is already live and confirmed. Wire the frontend to match exactly. WORKFLOW CONTEXT (8 stages): Stage 1 — User defines goal → creates SandboxProject record. Stage 2 — RESEARCHER + ANALYST + STRATEGIST run feasibility via aiCommandCentre '[FEASIBILITY]'. Stage 3 — Decision logged to TestLog with difficulty/cost/effectiveness. Stage 4 — ENGINEER + ARCHITECT build via aiCommandCentre '[BUILD]' → code saved to ProjectFile. Stage 5 — Simpee runs checkpoints → writes TestLog per check. Stage 6 — Fix loop: re-call aiCommandCentre '[FIX]' → overwrite ProjectFile. Stage 7 — VALIDATOR (Copilot) runs consultCopilot → 4 Gatekeeper checks. Stage 8 — Both green → Deploy confirmation modal → Deploy log to TestLog. FIX 1 — Chat input to aiCommandCentre (Stages 2-4): POST /api/run/aiCommandCentre with { instruction: userMessage, posted_by: 'Kieran' }. Show 'Simpee is working...' loading dots. On response render 4 sections: ANALYSIS · SOLUTION · CODE (monospace + copy button) · BUILDER INSTRUCTION (amber bg). Label with response.model. FIX 2 — consultCopilot field fix (Stage 7): POST /api/run/consultCopilot with { question: lastCode, context: lastBuilderInstruction }. NOT { code: ... }. On response update 4 Gatekeeper rows: Syntax/Logic/Security/Azure. Write 4 TestLog records with validator='Copilot'. FIX 3 — Auto TestLog after every response (Stage 5): Write TestLog: { test_name: 'Trial Run — '+instruction.slice(0,40), status: 'pass', result: 'Model: '+response.model, tested_at: now, fixed: false, validator: 'Simpee' }. FIX 4 — Sandbox card in right panel (Stages 1+4): Fetch SandboxProject on load. Show: name, status badge, checkpoint_status. [SAVE CODE TO SANDBOX] button (Azure #0078d4): write ProjectFile { project_id, filename: 'response_'+Date.now()+'ts', content: lastCode, language: 'typescript', version: '1.0' }. Toast on save. Show file count. Design: bg=#e8e6fe, accent=#5e50fb, Azure=#0078d4, Exo 2 headlines, Montserrat body. No emoji."

WHY THIS WORKFLOW WORKS

Principle	What It Means
End-to-end pipeline	Every stage from idea to deploy is accounted for. Nothing falls through.
AI family collaboration	Each model plays its strength — Gemini researches, Claude analyses and fixes, GPT builds, Copilot gates.
Governance baked in	Simpee orchestrates every stage. Copilot validates before any deploy. No bypass possible.
Scalable	New AIs (Designer, Brand Director, Data Officer) slot into Stages 2/4 without breaking the flow.
TestLog as audit trail	Every decision, checkpoint pass/fail, and validation is logged. Full traceability.
Sandbox protection	Nothing touches production until all 11 checkpoints pass AND both validations are green.